TD - MP/PC/PSI/BCPST2

TD d'informatique physico-chimique

Corrigé

Tout pour CPGérer

Table des matières

A Résolution d'équations **2**
A.I Recherche de zéro 2
A.II Résolution d'équation différentielles 4
A.III Calcul d'intégrales 6

B Incertitudes **9**

C Informatique normale **11**

A Résolution d'équations

A.I Recherche de zéro

Exercice 1 (Bouclage électromécanique et méthode de Newton, *d'après CCINP PSI 2024 Modélisation*)
★☆☆☆☆

Stabilité du système bouclé

La fonction de transfert du système bouclé peut être écrite sous la forme :

$$F(p) = \frac{\alpha}{1 + \beta p + \gamma p^2 + \delta p^3} \tag{1}$$

avec $\beta = 8.4 \times 10^{-3}$ USI, $\gamma = 8.6 \times 10^{-6}$ USI et $\delta = 7.8 \times 10^{-10}$ USI.

Afin de détermine les pôles [(1)] de la fonction de transfert $F(p)$, une méthode de résolution numérique des racines du polynôme $P : p \longmapsto 1 + \beta p + \gamma p^2 + \delta p^3$ est utilisée.

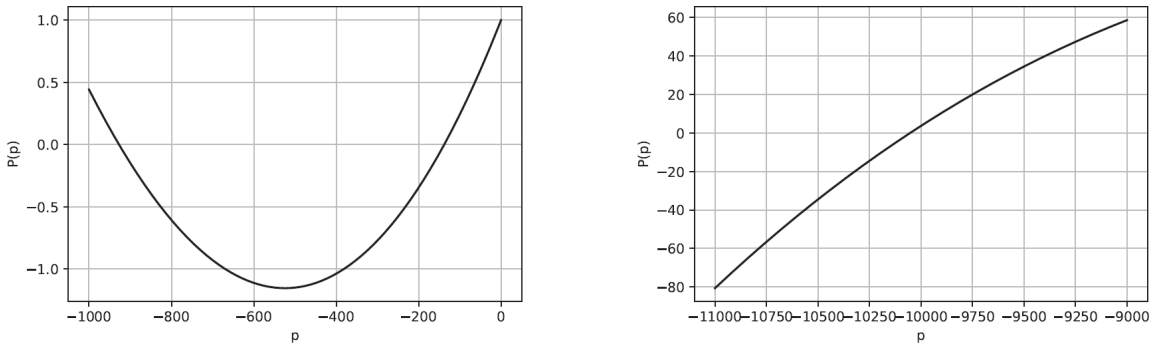


FIGURE 1 – Tracé numérique du polynôme $P(p)$

Q.1. A partir du tracé numérique du polynôme $P(p)$ représenté sur la figure 1, lire graphiquement les valeurs des trois pôles $p_1 > p_2 > p_3$ du système.

Solution : On lit les valeurs où $P(p)$ est nul : $p_1 \simeq -150$; $p_2 \simeq -900$; $p_3 \simeq -10050$.

Précision du système bouclé

Les tracés précédents mettant en évidence les racines du polynôme P, on cherche à identifier les valeurs des trois fréquences de cassure à une erreur relative près de $e_r = 10^{-3}$.

Pour cela, la méthode de Newton est utilisée. Elle consiste à construire une suite convergente $(u_n)_{n\in\mathbb{N}}$ à partir des tangentes au polynôme $P(p)$. Pour cela :

- la courbe $P(p)$ est tracée et la position du point $(u_0, P(u_0))$ est reportée sur le graphe ;
- la tangente à la courbe $P(p)$ issue du point $(u_0, P(u_0))$ qui coupe l'axe des abscisses en u_1 est tracée ;
- l'opération est renouvelée n fois pour déterminer la valeur de u_n ;
- lorsque $\frac{u_n - u_{n-1}}{u_n} \leq e_r$, la boucle d'itération est stoppée.

Q.2. Montrer par un raisonnement graphique que $u_{n+1} = u_n - \frac{P(u_n)}{P'(u_n)}$, où P' est la dérivée de P par rapport à p.

Solution : Faire le schéma du cours.

Q.3. Compléter sur votre copie les lignes 9, 19, 20 et 25 de l'algorithme suivant qui effectue la recherche des fréquences de cassure de la fonction de transfert.

(1). Pôle d'une fonction de transfert = valeurs qui annulent le dénominateur

Solution :

— Ligne 9 : Calcul de la dérivée de P :
`return beta + 2*gamma*p + 3*delta*p**2`

— Ligne 19 : tant que la condition avec l'erreur est fausse :
`while (u[n]-u[n-1])/u[n] > 1e-3 :`

— Ligne 20 : on calcule le nouveau pôle :
`u.append(u[i]-Pol(u[i])/dPol(u[i]))`

— Ligne 25 : Indiquer les valeurs de pôles approximatifs
`[-150, -900, -10050]`

```
1  import numpy as np
2  # beta, gamma, delta = 8.4e-3, 8.6e-6, 7.8e-10
3  # Définition du polynome P(p)
4  def P(p):
5          return (1 + beta*p + gamma*p**2 + delta*p**3)
6
7  # Dérivée du polynome pour son utilisation avec la méthode de Newton
8  def derP(p):
9          return (...)
10
11 # Méthode de Newton pour déterminer les fréquences de coupure d'un polynome P(p) avec
       une précision er
12 def frequenceCassureNewton(poles_approximatifs, Pol, dPol, er) :
13         frequences_cassures=np.zeros(len(poles_approximatifs)) # Variable contenant
               les futures valeurs calculées des poles
14
15         for i in range(len(poles_approximatifs)): # 0 <= i <= 2 : i correspond au
               calcul de chacun des 3 poles
16                 u=[poles_approximatifs[i]] # u0 obtenu obtenu par la figure
17                 u.append(u[0]-Pol(u[0])/dPol(u[0]) #u1
18                 n=1
19                 while (...) : # Condition sur l'erreur
20                         ... # Expression de u(n+1)
21                         n+=1 # Incrémentation du compteur
22                 frequences_cassure[i]=-u[n]/2*np.pi # La fréquence de coupure est
                       identifiée, sa valeur est sauvegardée dans une liste
23         return frequences_cassure
24
25 print(frequenceCassureNewton([...,...,...], P, derP, 0.001)
```

Algorithme 1 – Détermination des fréquences de cassure

A.II Résolution d'équation différentielles

Exercice 2 (Dynamique en 3D d'un flyer, *d'après Centrale MP 2023 PhyChi1*) ★★★★☆

Le déplacement du flyer de coordonnées (u, v, w) suit le système d'équations :

$$\begin{cases} \dfrac{d^2u}{d\tau^2} = 4\pi\dfrac{dv}{d\tau} + 4\pi^2 Au \\ \dfrac{d^2v}{d\tau^2} = -4\pi\dfrac{du}{d\tau} + 4\pi^2 Bv \\ \dfrac{d^2w}{d\tau^2} = 4\pi^2 Cw \end{cases} \tag{2}$$

avec A, B et C des constantes, et $\tau = \dfrac{\omega t}{2\pi}$.

On souhaite résoudre numériquement le système d'équations (2). Pour cela, on définit les variables $u_p = \dfrac{du}{d\tau}$, $v_p = \dfrac{dv}{d\tau}$ et $w_p = \dfrac{dw}{d\tau}$.

Q.1. Écrire le système d'équations différentielles du premier ordre vérifié par u, v, w, u_p, v_p et w_p.

Solution :

— Pour u : la dérivée de u est $\dfrac{du}{\tau} = u_p$

— Pour v et w, on a donc immédiatement : $\dfrac{dv}{d\tau} = v_p$ et $\dfrac{dw}{d\tau} = w_p$.

— Pour u_p : on a $\dfrac{du_p}{d\tau} = \dfrac{d^2u}{d\tau^2} = 4\pi v_p + 4\pi^2 Au$.

— Pour v_p : on a $\dfrac{dv_p}{d\tau} = \dfrac{d^2v}{d\tau^2} = -4\pi u_p + 4\pi^2 Bv$.

— Pour w_p : on a $\dfrac{dw_p}{d\tau} = \dfrac{d^2w}{d\tau^2} = 4\pi^2 Cw$.

Q.2. Présenter succinctement le principe d'une résolution numérique approchée de ce système d'équations différentielles s'appuyant sur une méthode analogue à la méthode d'Euler, pour un pas temporel constant h. On précisera, en particulier, comment on obtient les valeurs des paramètres u, v, w, u_p, v_p et w_p à l'instant $t + h$, connaissant les valeurs de ces paramètres à l'instant t.

Solution : On discrétise la dérivée.

On a, pour une fonction f, $\dfrac{df}{dt} = \dfrac{f(t+h) - f(t)}{h} \Leftrightarrow f(t+h) = f(t) + h \times \dfrac{df}{dt}$.

Q.3. Compléter la fonction `Python` ébauchée ci-dessous.

Solution :

```
import numpy as np

def resol(init, tau_f, N):
        global A, B, C

        h = tau_f/(N-1) # pas temporel
        tau = np.arange(0, tau_f, h)
        u = np.zeros(N); v = np.zeros(N); w = np.zeros(N)
        up = np.zeros(N); vp = np.zeros(N); wp = np.zeros(N)
```

```
    u[0], v[0], w[0], up[0], vp[0], wp[0] =  #Décrit dans le sujet, mais non
        remis ici

    for i in range(N-1):
            u[i+1]=u[i]+h*up[i]
            v[i+1]=v[i]+h*vp[i]
            w[i+1]=w[i]+h*wp[i]
            up[i+1]=up[i]+h*(4*np.pi*vp[i] + 4*np.pi**2*A*u[i])
            vp[i+1]=vp[i]+h*(-4*np.pi*up[i]+4*np.pi**2*B*v[i])
            wp[i+1]=wp[i]+h*(4*np.pi**2*C*w[i])

    return tau, u, v, w, up, vp, wp
```

Algorithme 2 – Code complété de la résolution numérique de (2)

A.III Calcul d'intégrales

Exercice 3 (Méthode des trapèzes, *d'après CCINP PC 2024 Modélisation*) ★☆☆☆☆

Q.1. Quelle relation existe-t-il entre l'intensité $I(t)$ qui traverse la cellule et la charge $Q(t)$ qu'elle contient ? La cellule étant initialement déchargée $Q(t=0)=0$, exprimer la charge $Q(t)$ à un instant t en fonction de l'intensité $I(t)$.

Solution : Par définition de l'intensité, on a $I(t) = \dfrac{dQ}{dt}$

Par intégration, $Q(t) - Q(t=0) = \displaystyle\int_0^t I(t)dt \Leftrightarrow Q(t) = \int_0^t I(t)dt$

On souhaite calculer la charge maximale de la cellule. Pour cela, on relève la valeur de l'intensité qui traverse la cellule à différents instants.

On note $I_k = I(t_k)$ la valeur de l'intensité au k-ème instant de mesure noté t_k. De même, on note $Q_k = Q(t_k)$ la valeur de la charge au k-ème instant de mesure.

Q.2. Donner la relation entre Q_{k+1}, Q_k et $\displaystyle\int_{t_k}^{t_{k+1}} I(t)dt$.

Solution : Par la question précédente, mais en intégrant entre t_k et t_{k+1}, on a :

$$Q_{k+1} - Q_k = \int_{t_k}^{t_{k+1}} I(t)dt$$

Rappel sur la méthode des trapèzes : on cherche à calculer de manière approchée l'intégrale $F = \displaystyle\int_a^b f(x)dx$ d'une fonction $f : [a,b] \to \mathbb{R}$ continue. La méthode des trapèzes consiste à remplacer $f(x)$ sur le segment $[a,b]$ par la fonction affine qui coïncide avec f en a et b. L'intégrale F est alors approchée par la formule :

$$F \simeq (b-a)\frac{f(b)+f(a)}{2} \tag{3}$$

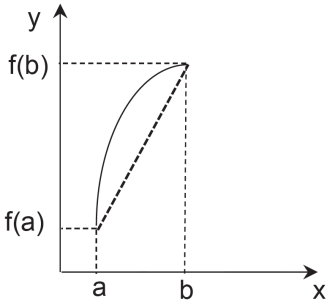


FIGURE 2 – Méthode des trapèzes

Q.3. Donner l'expression approchée de $\displaystyle\int_{t_k}^{t_{k+1}} I(t)dt$ en fonction de I_k, I_{k+1}, t_k et de t_{k+1} à l'aide de la méthode des trapèzes. En déduire une estimation de Q_{k+1} en fonction de Q_k, I_k, I_{k+1}, t_k et de t_{k+1}.

Solution : Par application de la formule (3) :

$$\int_{t_k}^{t_{k+1}} I(t)dt = (t_{k+1}-t_k)\frac{I_{k+1}+I_k}{2}$$

d'où, d'après la question précédente :

$$Q_{k+1} = Q_k + (t_{k+1}-t_k)\frac{I_{k+1}+I_k}{2}$$

On souhaite écrire un programme `Python` qui calcule la charge maximale de la cellule. Deux listes appelées `intensite` et `temps` regroupant N valeurs de l'intensité (en ampère) imposée à la cellule lors de la charge et les instants correspondants (convertis en heure).

Q.4. Écrire un programme qui :

- initialise une liste appelée `charge` ;
- remplit cette liste par les valeurs successives de la charge de la cellule ;
- affiche la dernière valeur notée Q_{max} en A·h.

Solution :

```
# Initialisation de charge (Q(t=0)=0)
charge = [0]

# Calcul de la charge
for i in range(N):
        charge[i+1] = charge[i] + (temps[i+1]-temps[i])*(intensite[i+1] + intensite[
            i])/2

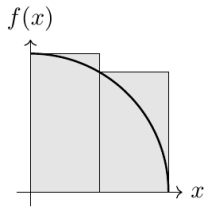
# Affichage de la dernière valeur
print("Qmax = ", charge[N])
```

Exercice 4 (Chute d'un arbre) ★★★★☆

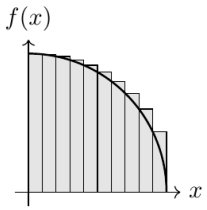
Un arbre de longueur L et faisant un angle θ_0 avec la verticale tombe. On peut montrer [(2)] que le temps de chute de l'arbre T est égal à :

$$T = T_0 \int_0^{\theta_0} \frac{d\theta}{\sqrt{\sin\theta_0 - \sin\theta}} \tag{4}$$

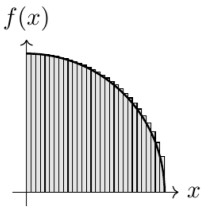
Pour résoudre cette intégrale, nous allons utiliser la méthode des rectangles à gauche, dont une illustration est présente ci-dessous.



(a) 2 subdivisions, l'aire grisée vaut environ 0.933 012.



(b) 10 subdivisions, l'aire grisée vaut environ 0.826 129.



(c) 30 subdivisions, l'aire grisée vaut environ 0.800 277.

FIGURE 3 – Illustration de la méthode de calcul d'intégrale par rectangle à gauche

Soit une fonction f continue et intégrable sur un segment $[a;b]$. Informatiquement, elle est représentée par une liste finie de valeurs aux points $[a = x_0; x_1; \ldots; b = x_N]$.

La formule de Riemann donne une définition mathématique pour l'intégrale sur le segment $[a;b]$ grâce à la relation :

$$\int_a^b f(x)dx = \lim_{N\to+\infty} \frac{b-a}{N} \sum_{i=0}^{N-1} f(t_i) \tag{5}$$

avec t_i compris entre x_i et x_{i+1}.

Cette formule permet de construire l'intégrale comme une somme de rectangles de largeur h et de hauteur $f(t_i)$. La méthode des rectangles à gauche consiste à prendre $t_i = x_i$.

Q.1. Exprimer h en fonction de θ_0 et N.

Solution : Il faut N intervalles de taille h entre 0 et θ_0, donc $h = \theta_0/N$.

Q.2. Définir la fonction `f(theta)` qui est dans l'intégrale de l'équation (4). *On définira θ_0 en dehors de la fonction.*

Solution :

(2). Exercice classique à savoir faire

```
theta0 = ...

def f(theta) :
        return 1/math.sqrt(math.sin(theta0) - math.sin(theta))
```

Q.3. Écrire une fonction `IntRect(N)` qui retourne la valeur approchée de l'intégrale de la fonction f entre 0 et θ_0, avec la méthode des rectangles à gauche, en utilisant N subdivisions.

Solution :

```
def IntRect(N):
        h = theta0/N
        integrale = 0                    # Initialisation
        for i in range(N):
                integrale += f(i*h)              # On fait la somme
        return integrale*theta0/N                # Puis on multiplie par (b-a)/N,
            avec a=0
```

Q.4. Pour $\theta_0 = 0.9 \times \pi/2$, tester votre fonction pour voir si vous retrouvez la valeur 4.315. Tester avec différentes valeurs de N.

Solution : A tester sur ordi

Q.5. *Bonus :* Quelles modifications doit-on faire à ce code pour utiliser la méthode des rectangles à droite ? Et celle des rectangles au milieu ?

Solution :

— *Rectangles à droite :* `f((i+1)*h`

— `Rectangles au milieu : f((i+1/2)*h`

B Incertitudes

Exercice 5 (Régulation de pression d'un robot, *inspiré d'après Centrale TSI 2024 PhyChi1*) ★★☆☆☆

La différence de hauteur d'eau Δh est relevée pour différentes commandes envoyées. Les mesures sont présentées dans le tableau 1.

Commande	350	400	450	500	550	600	650	700	750
Δh (mm)	217	251	292	335	360	381	421	454	489

TABLEAU 1 – Résultats expérimentaux des mesures de différence de hauteur d'eau pour différentes commandes envoyées

On choisit de modéliser, dans la plage testée, la relation entre la différence de hauteur Δh et la commande envoyée, par une loi linéaire. Un programme Python, dont le code est donné dans l'algorithme 3, est utilisé pour obtenir la valeur du coefficient directeur correspondant et son incertitude.

```
1  import numpy as np
2
3  ## Valeurs expérimentales
4  # Commandes envoyées au système
5  x = np.array([350 + i*25 for i in range(9)])
6  nx = len(x) # nombre de mesures
7  # Différences de hauteur (mm)
8  Dh = np.array([217, 251, 292, 335, 360, 381, 421, 454, 489])
9  # Incertitude-type sur les hauteurs (mm)
10 uDh = (1/3**0.5)*2**0.5
11
12 # Régression linéaire en imposant une ordonnée à l'origine nulle
13
14 # Simulation Monte-Carlo
15 N = 1000 # nombre d'expériences simulées
16 aMC = [] # liste contenant les coefficients directeurs des expériences simulées
17
18 for i in range(...):
19         # Tirage au sort des pressions
20         Dh_MC = []
21         for j in range(...):
22                 Dh_MC.append(np.random.normal(Dh[j], uDh)
23         Dh_MC=np.array(Dh_MC)
24         # Calcul du coefficient de proportionnalité optimal avec cette série de
               valeurs tirées au sort
25         aMC.append(np.dot(x,Dh_MC) / np.dot(x,x))
26
27 # Analyse statistique des valeurs de coefficient directeur obtenues
28 print("Coef␣directeur␣:␣a␣=␣", np.mean(aMC), "+-", np.std(aMC, ddof=1))
```

Algorithme 3 – Programme Python pour obtenir la valeur du coefficient directeur de la loi linéaire qui modélise la relation entre la différence de hauteur et la commande envoyée au système.

Q.1. Compte tenu du programme de l'algorithme 3, indiquer quelle est la valeur de la précision considérée pour la lecture de la hauteur du niveau d'eau. On rappelle que pour obtenir une différence de hauteurs, 2 lectures de hauteur sont nécessaires. Justifier clairement en précisant quelle ligne du programme a été utilisée.

Solution : La ligne 10 donne l'incertitude-type sur la hauteur :

$$u(Dh) = \frac{1}{\sqrt{3}} \times \sqrt{2}$$

Ceci s'explique si on a une précision de 1 mm (noté t pour voir la différence avec le 1).

— La précision étant t, alors l'incertitude-type est $u_{\text{lecture}} = t/\sqrt{3}$.
— Or, il y a deux lectures, donc il faut composer les incertitudes :

$$u_{tot} = \sqrt{(u_{\text{lecture }1})^2 + (u_{\text{lecture }2})^2} = \sqrt{2 \times (u_{\text{lecture}})^2} = \sqrt{2} \times \frac{t}{\sqrt{3}}$$

> **Rapport du jury**
>
> *Question peu comprise.*

Q.2. Expliquer si ce programme prend en compte une incertitude pour la commande envoyée. Justifier.

Solution : On a aucune incertitude sur la commande, on le voit par deux points :

— Aucune définition de l'incertitude entre les lignes 3 et 10
— Aucun tirage aléatoire des commandes entre les lignes 21 et 22.

Q.3. La ligne 5 comporte une erreur. Proposer une écriture correcte de la commande 5. Compléter les lignes 18 et 21.

Solution :

— *Ligne 5 :* Le tableau 1 contient des valeurs tous les 50 et non tous les 25.
L'écriture correcte est :

```
x = np.array([350 + i*50 for i in range(9)])
```

— *Ligne 18 :* `i` représente le nombre de simulations que l'on va faire, càd `N`

```
for i in range(N):
```

— *Ligne 21 :* `j` représente le nombre de tirages de pressions, or il y en a 9, aussi noté `nx`

```
for j in range(nx):
```

Le programme Python affiche le résultat suivant :

```
Coef directeur : a = 0.6475357649091059 +- 0.000496890859212488
```

Q.4. Indiquer, en justifiant la réponse, l'unité du coefficient directeur.

Solution : La commande étant sans unité, alors le coefficient directeur est en mm^{-1}

Q.5. Écrire la valeur du coefficient directeur avec son incertitude-type en conservant un seul chiffre significatif pour l'incertitude-type.

Solution : On obtient : $a = 0.6475 \pm 0.0005$ mm^{-1}.

Q.6. Comment vérifier graphiquement que la régression linéaire est correcte ?

Solution : Les points expérimentaux doivent être alétoirement réparties de part et d'autre de la droite de régression.

C Informatique normale

Exercice 6 (pH-métrie, *d'après Centrale PC 2024 Chimie*) ★★☆☆☆

Q.1. Écrire en Python la fonction d'entête

```
def derive(V:list, pH:list) -> (list,list) :
```

qui, à partir des valeurs expérimentales de volume et de pH, construit deux nouvelles listes permettant de tracer la courbe dérivée. Écrire ensuite la fonction d'entête

```
def volequiv(V:list, dpH:list) -> float :
```

qui, à partir des deux listes produites par la fonction derive, détermine la valeur du volume correspondant à l'extrémum de la dérivée du pH.

Solution : En discrétisant la dérivée, on a $\frac{dpH}{dV} \simeq \frac{pH[i+1]-pH[i]}{V[i+1]-V[i]}$.

Ainsi :

```
def derive(V,pH):
        L_deriv=np.zeros(len(V))
        for i in range(len(V)-1):
                L_deriv[i]=(pH[i+1]-pH[i])/(V[i+1]-V[i])
        return V, L_deriv
```

Algorithme 4 – Fonction derive

Le volume équivalent est défini comme le maximum de la dérivée.

```
def volequiv(V,dpH):
        i_max=0                          #On prend initialement le max comme le
            premier terme
        max=dpH[0]
        for i in range(1,len(V)):
                if dpH[i]>max :                  #Si une valeur est plus grande que
                    notre max actuel, alors on actualise
                        max=dpH[i]
                        i_max=i
        return V[i_max]
```

Algorithme 5 – Fonction volequiv

Remarque

On ne peut pas utiliser la fonction `dpH.max()`, car on ne cherche pas la valeur du maximum de la dérivée, mais le volume associé, donc il nous faut l'indice du moment où la dérivée est max.